Lumni Coworking

Guía de estudio del proyecto

Prueba técnica Full Stack · Estado implementado: Partes 1.1 a 2.2

Esta guía explica qué hace cada capa, cómo se conectan y qué decisiones conviene poder defender en una sustentación. Está escrita para estudiar el código de este repositorio, no como una introducción genérica a React o Node.

Ruta de lectura

1. Empieza por el recorrido de una solicitud: interfaz -> API -> servicio -> PostgreSQL.
2. Revisa las cinco funcionalidades en orden: cotizador, reserva, administración, correo y chat.
3. Ejecuta las pruebas y abre las rutas mencionadas mientras lees.
4. Cierra practicando las preguntas de sustentación del final.

1. Mapa del sistema

El proyecto separa interfaz, API y persistencia:

```
Navegador (React + TypeScript)
        | /api/*
        v
Proxy de Vite durante desarrollo
        |
        v
Express + validación Zod
        |
        +-- servicios de dominio TypeScript
        +-- PostgreSQL mediante pg
        +-- worker de correos en segundo plano
        +-- cliente OpenAI-compatible -> llama.cpp local
```

React presenta los estados y recoge los datos. Express valida cada entrada; los servicios resuelven reglas de negocio. PostgreSQL conserva precios pactados, estados, sesiones, jobs y conversaciones. El navegador no calcula precios ni decide si una sala está disponible.

Archivos para ubicarse

- src/App.tsx: cotizador, navegación, sesión y apertura de los flujos.
- src/BookingWizard.tsx: reserva en tres pasos.
- src/AdminDashboard.tsx: administración de reservas.
- src/AdminEmailJobs.tsx: bandeja local de correos.
- src/AssistantChat.tsx: conversación con el asistente.
- src/contracts.ts: tipos de las respuestas que comparte el frontend.
- server/src/app.ts: rutas HTTP, validación y autorización.
- server/src/quote.ts: cálculo puro de precio y horario.
- server/src/quote-service.ts: disponibilidad y sugerencias usando un repositorio.
- server/src/booking-service.ts: borradores, servicios y confirmación.
- server/src/booking-state.ts: transiciones permitidas.
- server/src/email-worker.ts: cola, reintentos y captura local.
- server/src/assistant-chat-service.ts: orquestación del chat y creación del borrador.
- server/src/assistant-extraction.ts: schema y extracción determinista de evidencia.
- server/prompts/reservation-assistant.md: instrucciones del modelo.
- db/migrations/: esquema versionado de PostgreSQL.

2. Parte 1.1: cotizador

Recorrido de una cotización

1. La pantalla carga las salas con GET /api/rooms.
2. El cliente envía roomId, date, startTime y durationHours a POST /api/quotes.
3. Zod comprueba el formato de la solicitud; la lógica de quote.ts valida fecha, duración y horario.
4. quote-service.ts lee la tarifa actual de PostgreSQL y comprueba cruces con reservas confirmed.
5. Si el tramo no está libre, prueba horarios posteriores en intervalos de 30 minutos, hasta el cierre.
6. React presenta disponibilidad, razón, desglose y, cuando existe, el siguiente inicio posible.

Precio

El cálculo se concentra en calculateQuotePrice:

```
base = tarifa_hora × duración
neto = base - descuento
recargo_sábado = neto × 25 %   (solo sábado)
total = neto + recargo_sábado
```

Tarifas por hora: pequeña $40.000, mediana $70.000 y grande $120.000 COP. La duración válida es de 1 a 10 horas, en bloques de media hora. Se descuenta 10 % desde 4 horas y 20 % desde 8 horas. El horario es lunes a sábado, 07:00–21:00; domingo cerrado.

La fecha de pantalla se interpreta en Bogotá (UTC-05:00) y los instantes se guardan como timestamptz. La comparación de intervalos usa la convención semiabierta [inicio, fin): una reserva puede empezar justo cuando otra termina.

Por qué hay una regla en PostgreSQL además del chequeo

La consulta de disponibilidad ayuda a cotizar, pero no bloquea la sala. Otra persona puede confirmar entre la cotización y la confirmación. La restricción GiST de la migración inicial prohíbe traslapes entre reservas confirmadas de la misma sala, incluso si dos transacciones compiten. La cotización es informativa; la confirmación vuelve a ser la operación decisiva.

Pruebas: server/src/quote.test.ts cubre precios y horarios; server/src/quote-service.test.ts cubre disponibilidad y sugerencias.

3. Parte 1.2: reserva por pasos

El wizard guarda cliente, sala/horario y servicios. Cada paso llama al backend; no depende solo del estado de React. El current_step persiste el avance y el servidor impide saltar pasos o guardar servicios antes de tener cliente y horario.

Reanudación y privacidad

Al crear un borrador, el servidor genera un token aleatorio. El navegador guarda bookingId y el token en localStorage para retomarlo después de cerrar o recargar. PostgreSQL guarda únicamente el SHA-256 del token. Las rutas del borrador requieren Authorization: Bearer <token> y solo consultan ese bookingId; no hay una ruta que permita enumerar los borradores ajenos.

Este token es distinto de la sesión de login de la Parte 1.3 y del token del chat de la Parte 2.2. Cuando el asistente crea el borrador, reutiliza de forma intencional el token privado de su conversación para enlazar ambos recursos; la reserva queda visible solo desde ese chat y el wizard.

Confirmación idempotente y concurrencia

La transición se valida en transitionBooking: draft -> confirmed, confirmed -> completed/cancelled y draft -> cancelled. Los estados terminales no se reabren. Repetir confirmed -> confirmed devuelve el mismo estado.

confirmBooking bloquea la fila con SELECT ... FOR UPDATE, valida el estado y actualiza dentro de una transacción. No vuelve a calcular total: lee el precio ya guardado. Para dos borradores distintos que confirman el mismo espacio y horario, la restricción GiST de PostgreSQL deja ganar a uno y el otro recibe 409.

Servicios y precios decididos

La guía pide proyector y café pero no define sus precios. Este proyecto documenta la decisión de $50.000 COP por proyector y $8.000 por persona de café. Al cambiar los servicios, las líneas previas se reemplazan en vez de sumarse de nuevo.

4. Parte 1.3: acceso y administración

db:seed crea un cliente y un administrador de demostración. Las contraseñas se hashean con scrypt; nunca se guardan en texto plano. El login entrega un token aleatorio de ocho horas y PostgreSQL conserva su hash en auth_sessions. Logout marca la sesión como revocada.

El middleware requireAdmin valida la sesión y el rol en el backend. Ocultar el botón en React no sería autorización suficiente. Un cliente puede iniciar sesión, pero recibe 403 en /api/admin/*.

El panel ofrece filtro por estado y sala, paginación con tamaño máximo de 50 y detalle con desglose. La cancelación requiere motivo de 1–500 caracteres; una transacción bloquea la reserva, valida el cambio de estado y guarda cancellation_reason y cancelled_at.

Credenciales locales (solo desarrollo):

- Cliente: cliente@lumni.local / Cliente-demo-2026!
- Administrador: admin@lumni.local / Admin-demo-2026!

No se deben reutilizar en producción.

5. Parte 2.1: correo en segundo plano

La transición inicial a confirmed inserta un job en booking_email_jobs dentro de la misma transacción de confirmación. booking_id es único: la misma reserva no genera un segundo job aunque se confirme dos veces. La ruta HTTP solo guarda el trabajo y responde; el envío lo hace un worker independiente del request.

El worker reclama jobs con FOR UPDATE SKIP LOCKED, así dos procesos no toman el mismo correo. La captura local se escribe en captured_emails, con una restricción única por job_id, para que el resultado sea inspeccionable sin proveedor externo y una repetición no duplique la entrega local.

La especificación se interpreta como un envío inicial más cinco reintentos: seis intentos totales. Las esperas crecen 1, 2, 4, 8, 16 segundos. Tras el sexto fallo, el job queda failed con el último error. Un job processing cuyo lock lleve más de cinco minutos se recupera al arrancar y periódicamente. La pestaña Correos del panel admin permite filtrar por estado e inspeccionar destinatario, cuerpo, intentos, error y captura.

6. Parte 2.2: asistente con modelo local

Configuración de llama.cpp

El servidor usa el SDK OpenAI contra un endpoint configurable. En .env:

```
AI_BASE_URL=http://localhost:11434/v1
AI_API_KEY=local-llama
AI_MODEL=local-model
AI_TIMEOUT_MS=120000
```

AI_MODEL debe ser exactamente un id devuelto por GET http://localhost:11434/v1/models. La clave es un placeholder para un servidor local sin autenticación. El .env está ignorado por Git; .env.example no contiene la ruta privada del modelo del equipo.

Separación de responsabilidades

El modelo no es autoridad sobre el negocio. reservation-assistant.md le permite extraer fragmentos del mensaje actual, pero no calcular precios, consultar disponibilidad, inventar datos, acceder a otros clientes ni confirmar/cancelar. La salida tiene un schema Zod cerrado; campos extra como price o confirm se rechazan.

El servidor comprueba que cada fragmento de evidencia esté presente en el mensaje normalizado (sin distinguir mayúsculas ni acentos). chrono-node en español resuelve fechas relativas con la fecha actual de Bogotá como referencia. Las reglas locales convierten horas y duraciones; “en la mañana” no se convierte en una hora exacta. La capacidad no existe en el catálogo, así que el asistente no la inventa ni elige una sala por cantidad de personas.

Si falta tamaño, fecha, inicio o duración, el servidor pregunta solo por el primer dato faltante. Con esos datos llama a createQuote, que consulta la misma API de cotización que usa el resto de la app. Si no hay disponibilidad, conserva la hora sugerida en un campo separado: no se convierte en preferencia hasta que el usuario acepte claramente o proponga otra hora.

Cuando los datos requeridos están presentes, el servidor crea un borrador con cliente, asistentes y servicios. El token del chat se comparte con ese borrador para que el wizard pueda retomarlo. El chat no llama al endpoint de confirmación; la persona debe abrir el wizard y pulsar Confirmar reserva.

Conversación privada y reanudable

POST /api/assistant/chats crea una sesión de 30 días y devuelve un token aleatorio. PostgreSQL almacena su hash, el contexto validado y los mensajes. El navegador conserva el token en localStorage. Para procesar un turno, el modelo recibe el prompt, el catálogo público de salas y el mensaje actual; no se le envía el historial completo ni información de otros clientes. El contexto previo se conserva y valida en el servidor.

7. Cómo ejecutar y verificar

Desde PowerShell, en la raíz:

```
Copy-Item .env.example .env
# Ajusta AI_MODEL al id de llama.cpp en tu equipo.
docker compose up -d db
npm install
npm run db:migrate
npm run db:seed
npm run dev
```

Pantalla: http://localhost:5173. API: http://localhost:3001. El modelo debe responder en la URL de AI_BASE_URL antes de enviar el primer mensaje.

Verificaciones automatizadas:

```
npm test
npm run lint
npm run build
```

En este estado del repositorio se verificaron 36 pruebas unitarias, build y lint. Además se probó con el llama.cpp local: frase en español -> campos explícitos -> cotización del servidor -> borrador recuperable -> apertura del wizard. No se pulsó confirmar desde el chat.

8. Preguntas para practicar la sustentación

1. ¿Por qué una cotización libre no garantiza el espacio y qué impide dos confirmaciones simultáneas?
2. ¿Cómo demuestras que repetir la confirmación no recalcula el precio ni duplica el correo?
3. ¿Qué impide que un cliente invoque la API administrativa cambiando el frontend?
4. ¿Qué pasa con un job de correo si el worker cae después de reclamarlo?
5. ¿Por qué el modelo recibe evidencia y no se le pide devolver un precio final?
6. ¿Cómo evitas que “el sábado” dependa de la zona horaria del navegador?
7. ¿Qué ocurre si la hora sugerida no es aceptada explícitamente?
8. ¿Qué credenciales o datos nunca deben ir al prompt ni al repositorio?

9. Vocabulario del proyecto

- Outbox: tabla transaccional que guarda trabajo pendiente junto con el cambio de negocio que lo origina.
- Bearer token: credencial privada que autoriza a quien la presenta; se almacena como hash en PostgreSQL.
- Idempotencia: repetir una operación produce el mismo estado sin duplicar efectos.
- FOR UPDATE: bloqueo de fila que serializa cambios concurrentes sobre una reserva.
- SKIP LOCKED: permite que varios workers reclamen filas diferentes sin esperar por el mismo job.
- Schema Zod: contrato ejecutable que valida datos no confiables en runtime.
- Evidencia textual: fragmento que el modelo señala, pero que el servidor vuelve a verificar antes de usarlo.
- timestamptz: instante normalizado que se compara correctamente aunque la entrada venga en hora local de Bogotá.